

आलु

- समय सीमा: 10 minutes
- वातावरण: एउटा GPU (≈16 GB VRAM), इन्टरनेट छैन
- समाधानको आकार: `solution.ipynb` ≤ 1 MB
- भण्डारण: 5 GB

कार्य

तपाईंको साथीले अनुमान गर्ने खेल खेल सुभाब दिन्छ। उसले निर्णायकका रूपमा निश्चित शब्दावलीबाट एउटा गोप्य शब्द छान्छ, र तपाईंले त्यसलाई बढीमा 30 पालोभित्र पत्ता लगाउनुपर्छ। प्रत्येक पालोमा निर्णायकले दुई शब्द तुलना गर्छ र तीमध्ये कुनचाहिँ अर्थगत रूपमा गोप्य शब्दसँग बढी नजिक छ भनेर बताउँछ। प्रत्येक खेल निश्चित जोडी `lamp vs potato` बाट सुरु हुन्छ, किनकि ती तपाईंको साथीका मनपर्ने दुई कुरा हुन्। त्यसपछि तपाईंको प्रोग्रामले एउटा नयाँ शब्द प्रस्ताव गर्छ। तुलनाको विजेता कायम राखिन्छ र तपाईंको अर्को प्रस्तावसँग तुलना गरिन्छ। तपाईंले गोप्य शब्द व्याक्कै प्रस्ताव गर्नेबित्तिकै खेल जित्नुहुन्छ। मिलानमा ठूला-साना अक्षरको भेद गरिंदैन। तपाईंले प्रस्ताव गर्ने प्रत्येक शब्द `dataset/vocabulary.json` मा हुनैपर्छ।

प्रोटोकल र डेटा लोडिङसहितको पूर्ण उदाहरण `solution.ipynb` मा छ। तपाईं `PublicEmbeddingPlayer` class परिवर्तन गर्न सक्नुहुन्छ। तपाईंको प्रोग्राम एकपटक प्रारम्भ गरिन्छ र एउटै रनमा प्रत्येक खेल खेल्छ; प्रोटोकलले प्रत्येक खेलको सुरुमा नयाँ `PublicEmbeddingPlayer` सिर्जना गर्छ।

निर्णायक

तपाईंको प्रोग्रामले निर्णायकलाई एउटा JSON object पठाउँछ र निर्णायकले एउटा JSON object मार्फत जवाफ दिन्छ।

प्रोटोकल व्याख्या गर्न मात्र गोप्य शब्द देखाइएको एउटा पूर्ण उदाहरण:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}
<- matches: game won
```

पालोहरू 1 देखि 30 सम्म अनुक्रमित छन्।

`verdict` का विकल्पहरू `first` हुन्, जसको अर्थ `word1` बढी नजिक छ; `second` हुन्, जसको अर्थ `word2` बढी नजिक छ; वा `same` हुन्, जसको अर्थ दुवै शब्द गोप्य शब्दसँग समान रूपमा नजिक छन्।

`winner_word` अर्को तुलनाका लागि कायम राखिएको शब्द हो। `same` निर्णयमा पहिलो शब्द नै कायम रहन्छ।

Dataset

प्रत्येक `split` मा साभाना:

- `dataset/vocabulary.json` — 1602 वटा अद्वितीय lowercase शब्दहरू। गोप्य शब्द सधैं यिनैमध्ये एउटा हुन्छ।
- `dataset/public_embeddings.npy` — `float32`, `shape` (1602, 2560)। Row `i` शब्दावलीको शब्द `i` सँग सम्बन्धित छ। यी *सार्वजनिक* embeddings हुन्; निर्णायकले फरक, निजी representation प्रयोग गर्छ।

`split` हरू गोप्य शब्दका set हुन्:

Split	शब्दहरू	उत्तरहरू	यसको प्रयोग
<code>test_public</code>	120	✓ <code>dataset/test_public.json</code>	आफ्नो समाधान चलाउन र आफैं score गर्न
<code>test_leaderboard_a</code>	120	गोप्य	live leaderboard
<code>test_leaderboard_b</code>	120	गोप्य	अन्तिम ranking

`train split` छैन — labelled rows बाट केही पनि fit गरिंदैन।

उपलब्ध गराइएका models

दुईवटा pretrained embedding models यस कार्यसँगै उपलब्ध छन् र प्रयोग गर्न सकिन्छ:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

दुवैलाई तिनको local path बाट load गर्नेपर्छ; उदाहरणका लागि Hugging Face hub id "BAAI/bge-m3" ले download सुरु गर्छ र असफल हुन्छ, किनकि मूल्याङ्कन offline हुन्छ। प्रत्येक directory मा offline call देखाउने चलाउन मिल्ने `example.py` छ।

उपलब्ध libraries: `numpy`, `torch`, `sentence-transformers`। इन्टरनेट छैन, downloads छैनन्, अन्य packages छैनन्।

Output

केही छैन। यो interactive कार्य हो: तपाईंको समाधानले कुनै answer file लेख्दैन; यसले माथि वर्णन गरिए अनुसार stdin/stdout मार्फत निर्णायकसँग सञ्चार गर्छ।

Metric

पालो t मा फेला पारिएको खेलले $1.0 - 0.02 \times \max(0, t - 10)$ score प्राप्त गर्छ; 30 पालोभित्र समाधान नभएको खेलले 0 score प्राप्त गर्छ। त्यसैले पालोहरू 1-10 ले 1.00, पालो 20 ले 0.80, र पालो 30 ले 0.60 score प्राप्त गर्छन्।

तपाईंको कार्यको score भनेको औसत $\text{game score} \times 100$ हो, जुन 0.00 र 100.00 को बीचमा हुन्छ।

10-minute सीमा start-up, तयारी र test set का सबै 120 खेल समेट्ने एउटै budget हो।

कसरी submit गर्ने

1. `solution.ipynb` खोल्नुहोस्, `PublicEmbeddingPlayer` edit गर्नुहोस्, र यसले काम गरिरहेको सुनिश्चित गर्न सबै cells चलाउनुहोस्।
2. वैकल्पिक रूपमा, यसलाई locally जाँच्नुहोस्: `python local_test.py solution.ipynb --limit 5`। local निर्णायकले *सार्वजनिक* embeddings प्रयोग गर्छ, त्यसैले यसको score मार्गदर्शन मात्र हो।
3. `solution.ipynb` save गर्नुहोस्।
4. JupyterLab को बायाँ sidebar मा रहेको Git tab खोल्नुहोस्।
5. `solution.ipynb` stage गर्नुहोस् (यसको छेउमा रहेको + icon)।
6. commit message प्रविष्ट गर्नुहोस् र Commit मा click गर्नुहोस्।
7. push गर्न माथितिर फर्किएको तीर भएको cloud मा click गर्नुहोस्।
8. यस Contest page मा फर्कनुहोस् र तपाईंले दिएको commit message सँग मिल्ने गरी Submit मा click गर्नुहोस्।

आवश्यक सबै तयारी र inference समेट्ने, `solution.ipynb` नाम भएको ठ्याक्कै एउटा file submit गर्नुहोस्।